

# GraphRagX: Graph Neural Retrieval and Large Language Model Reasoning for Knowledge Graph Question Answering

Sonja Gievska\*, Martina Toshevska\*,  
Andrea Stevanoska\*, Viktor Kostadinovski\*

\*Faculty of Computer Science and Engineering, Ss. Cyril and  
Methodius University in Skopje

## Abstract

Knowledge Graph question answering requires systems to answer natural language questions using structured facts represented as entities and relations. This project investigates a hybrid approach for question answering over WebQSP local knowledge graph subgraphs by combining graph neural retrieval with large language model based answer generation. The framework uses a question-aware graph neural network to rank candidate answer entities and then extracts compact path-based evidence from the graph for the selected candidates. This evidence is verbalized as triples and provided to a large language model, which produces the final answer and an explanation grounded in the extracted paths. The goal of the project is to study how structured graph retrieval and explicit reasoning evidence can support more interpretable question answering over knowledge graphs. The implementation, experimental configurations, and evaluation results are publicly available at <https://github.com/Adstract-AI/graphragx>.

## 1 Introduction

Knowledge graph question answering (KGQA) aims to answer natural language questions using structured information represented as entities and relations. Unlike open-domain question answering over text, this task requires mapping the question to relevant graph elements and identifying the entities, relations, and paths necessary to produce the answer.

**Knowledge Graphs (KG)** encode information as triples of the form (head, relation, tail), enabling explicit reasoning over connections between entities. In the context of KGQA, the goal is to use this structured representation to locate answer entities through multi-hop relational paths, rather than relying on unstructured text.

**Graph Neural Networks (GNNs)** learn node representations through message passing over graph structure, allowing models to capture both local and multi-hop dependencies. This makes them well suited for reasoning over knowledge graphs, where relevant information is often distributed across multiple connected entities.

**Large Language Models (LLMs)** operate on unstructured text and are designed for natural language understanding and generation. While they excel at producing fluent answers, they do not inherently reason over structured graph data, which limits their ability to perform precise multi-hop reasoning in knowledge graphs.

A central challenge in KGQA is retrieving relevant information from the graph. Question-specific subgraphs often contain both the correct answer and many irrelevant connections, making it necessary to balance completeness and noise. Effective systems must therefore rank candidate answer entities while preserving enough relational structure to support reasoning.

This motivates combining graph-based reasoning with language models. Graph neural networks can be used to identify relevant entities and extract structured evidence, while large language models can interpret this evidence and generate natural language answers. However, connecting these components remains challenging, as graph structure must be transformed into a form suitable for language model input without losing important relational information.

In this work, we study a modular KGQA pipeline that separates graph-based retrieval from language-model-based answer generation. A question-aware GNN is used to identify candidate answer entities from local WebQSP subgraphs, after which compact path-based evidence is extracted and provided to the LLM. This design preserves explicit graph structure, reduces irrelevant context, and enables answers that can be traced back to interpretable reasoning paths.

## 2 Related Work

Knowledge graph question answering (KGQA) has been studied extensively, with early approaches focusing on mapping natural language questions to structured queries over knowledge bases. A key step in this direction was the introduction of WebQuestionsSP by Yih et al. [1], which reformulates question answering as a semantic parsing problem and provides a benchmark for reasoning over Freebase-style knowledge graphs. This dataset has since become a standard testbed for evaluating graph-based reasoning methods.

More recently, retrieval-augmented generation (RAG) has emerged as a powerful paradigm for improving factual accuracy in language models by conditioning generation on external evidence. The original formulation by Lewis et al. [2] focuses on retrieving unstructured text, but its core idea—combining retrieval with generation—has motivated extensions to structured domains such as knowledge graphs.

In parallel, research has explored how to retrieve and expand relevant context for question answering across multiple sources. For example, PullNet [3] iteratively gathers evidence from both knowledge bases and text, demonstrating the benefit of dynamically constructing a reasoning context. While effective, such approaches do not leverage recent advances in large language models or explicitly model graph structure through neural message passing.

Another line of work relies on embedding-based representations of knowledge graphs. Methods such as EmbedKGQA [4] use learned embeddings to perform multi-hop reasoning. Although these approaches are computationally efficient, they often abstract away explicit reasoning paths, making it more difficult to provide interpretable explanations for predicted answers.

To address this limitation, several works have focused on reasoning directly over graph structure.

Models such as TransferNet [5] and ReaRev [6] perform question-guided propagation over local graph neighborhoods, allowing them to capture multi-hop dependencies in a more interpretable manner. These approaches are closely aligned with the graph reasoning component of our method.

More recently, hybrid approaches have begun to combine graph reasoning with large language models. GNN-RAG [7] integrates graph neural retrieval with LLM-based reasoning by identifying relevant nodes and extracting supporting paths. Similarly, G-Retriever [8] extends retrieval-augmented generation to graph-structured data by retrieving compact graph evidence rather than unstructured passages. In a related direction, Rel-LLM [9] studies how language models can be enhanced with relational information for graph-based reasoning.

Our work follows this line of research in a focused WebQSP setting. Rather than treating the local graph only as text or relying only on graph embeddings, the system separates candidate answer retrieval, path-based evidence extraction, and final answer generation. This separation makes it possible to evaluate the retrieval quality of the graph model, the usefulness of the extracted evidence, and the correctness of the final language-model answer as distinct parts of the KGQA pipeline.

### 3 Solution Architecture

#### 3.1 Problem Definition

Knowledge graph question answering (KGQA) aims to answer natural language questions using facts stored in a structured knowledge graph. A knowledge graph is typically represented as a directed multi-relational graph

$$G = (\mathcal{V}, \mathcal{E}, \mathcal{R}),$$

where  $\mathcal{V}$  is the set of entities,  $\mathcal{R}$  is the set of relation types, and each edge

$$(u, r, v) \in \mathcal{E}$$

represents a factual relation  $r \in \mathcal{R}$  between entities  $u, v \in \mathcal{V}$ .

Given a natural language question  $q$  and a graph  $G$ , the objective of KGQA is to identify the entity or set of entities

$$A_q \subseteq \mathcal{V}$$

that correctly answer the question. In practical settings, however, the full graph may contain millions of entities and relations, making direct reasoning over the entire graph computationally infeasible.

**Reasoning over Large KGs.** Large language models possess strong general reasoning capabilities, but they are limited when answering questions that depend on highly specific or structured relational knowledge. Since this information may not be reliably stored in the model parameters, directly prompting an LLM can lead to hallucinations or incomplete answers.

Retrieval-augmented generation (RAG) addresses this limitation by retrieving external evidence relevant to the input question and conditioning the language model on that evidence during inference. While traditional RAG systems operate primarily over unstructured text, KGQA

requires retrieval over graph-structured knowledge sources. Instead of retrieving documents or passages, the system must identify a compact subgraph containing the entities and relations necessary for answering the question.

This retrieval problem is challenging because local graph neighborhoods often contain many irrelevant entities and relations. Consequently, effective KGQA requires not only finding answer candidates, but also filtering the graph structure so that only the most relevant relational evidence is preserved for downstream reasoning.

**Question-Guided Graph Retrieval.** Formally, given a question  $q$ , the goal is not to process the entire graph  $G$ , but rather to retrieve a smaller question-specific subgraph

$$G_q = (\mathcal{V}_q, \mathcal{E}_q),$$

where

$$\mathcal{V}_q \subseteq \mathcal{V}, \quad \mathcal{E}_q \subseteq \mathcal{E},$$

such that  $G_q$  retains the relational evidence necessary for recovering the correct answer entities.

The retrieval stage can therefore be viewed as a ranking problem over graph nodes. For each candidate entity  $v \in \mathcal{V}_q$ , the model estimates a relevance score

$$s(v, q),$$

which measures how likely the entity is to answer the question. The objective is to rank gold answer entities higher than irrelevant nodes while preserving the graph paths that justify the prediction.

This formulation naturally motivates the use of Graph Neural Networks (GNNs). Through message passing and neighborhood aggregation, GNNs can propagate question-relevant information across local graph structure and capture multi-hop dependencies between entities. Rather than treating the graph as flat text, the model explicitly reasons over relational connectivity, allowing retrieval to remain grounded in graph structure.

**Project Objective.** The objective of this project is to design a modular KGQA pipeline that combines graph-based retrieval with language-model-based reasoning. The system first uses a question-aware GNN to identify and rank likely answer entities within a local graph neighborhood. It then extracts compact reasoning paths connecting question entities to the retrieved candidates and converts this structured evidence into textual form for the language model.

By separating graph retrieval from final language generation, the proposed approach aims to:

- reduce irrelevant graph context,
- preserve explicit relational structure,
- support multi-hop reasoning over local subgraphs,
- and improve interpretability by grounding generated answers in extracted graph evidence.

This formulation follows the broader direction of graph-aware retrieval-augmented generation, where structured retrieval serves as the foundation for reliable reasoning over large relational knowledge bases.

### 3.2 Dataset and Graph Representation

To evaluate the proposed retrieval and reasoning framework, we use the WebQuestionsSP (WebQSP) [1] dataset, a benchmark designed for question answering over large-scale knowledge graphs. WebQSP extends the original WebQuestions dataset by providing structured semantic parses and explicit grounding of questions to entities and relations in Freebase.

**WebQSP Dataset.** Each sample in WebQSP consists of a natural language question  $q$ , one or more topic entities identified in the question, a set of gold answer entities, and a local question-specific subgraph extracted from Freebase. This local subgraph contains entities and relations surrounding the topic entities and serves as the retrieval space for graph-based reasoning.

Questions in WebQSP often require compositional or multi-hop reasoning, where the correct answer depends on traversing multiple relations between entities. The underlying knowledge source is Freebase, which can be represented as a directed multi-relational graph:

$$G = (\mathcal{V}, \mathcal{E}, \mathcal{R}).$$

**Graph Construction.** To support graph neural retrieval and subgraph reasoning, the raw knowledge graph is transformed into a graph representation compatible with Graph Neural Networks. The graph is modeled as a heterogeneous relational graph where entities correspond to nodes and Freebase relations correspond to typed directed edges.

Formally, each entity is assigned a unique integer identifier through a node indexing function:

$$\phi : \mathcal{V} \rightarrow \{0, 1, \dots, |\mathcal{V}| - 1\}.$$

This mapping enables efficient tensor-based processing within graph learning frameworks such as PyTorch Geometric. After indexing, the graph connectivity is represented using the edge index structure:

$$\text{edge\_index} \in N^{2 \times |\mathcal{E}|},$$

where each column

$$\begin{bmatrix} i \\ j \end{bmatrix}$$

represents a directed edge from source node  $i$  to target node  $j$ .

In parallel, relation types are stored separately using an edge-type vector:

$$\text{edge\_type} \in N^{|\mathcal{E}|},$$

where each value corresponds to a relation identifier associated with the respective edge. This separation allows the model to distinguish between different semantic relationships during message passing.

**Question and Entity Processing.** For every question  $q$ , the dataset provides one or more topic entities that act as entry points into the graph. These entities are mapped to their corresponding node indices and used to initialize the retrieval process.

To support supervised training of the retriever, gold answer entities are also converted into graph node indices:

$$A_q = \{a_1, a_2, \dots, a_n\},$$

where each

$$a_i \in \mathcal{V}$$

corresponds to a correct answer node in the graph.

The resulting training instance therefore contains:

- the textual question,
- indexed topic entities,
- indexed answer entities,
- graph connectivity information,
- and relation-type metadata.

This representation enables efficient neighborhood expansion, subgraph extraction, and graph-based retrieval during both training and inference.

**Subgraph Extraction for Retrieval.** Since the full knowledge graph contains millions of nodes and relations, directly processing the entire graph is computationally impractical. Instead, for each question we construct a local candidate subgraph centered around the topic entities.

Given a set of seed nodes

$$S_q \subseteq \mathcal{V},$$

the system performs bounded neighborhood expansion up to  $k$  hops:

$$\mathcal{N}^{(k)}(S_q),$$

producing a compact question-specific subgraph:

$$G_q = (\mathcal{V}_q, \mathcal{E}_q).$$

This local graph serves as the retrieval space for the GNN retriever. Restricting reasoning to a bounded neighborhood significantly reduces computational complexity while preserving most of the relational evidence required for answering multi-hop questions.

The resulting graph representation therefore forms the structural foundation of the entire pipeline, enabling graph neural retrieval, reasoning-path extraction, and downstream language-model-based answer generation.

### 3.3 Question-Aware GNN Reasoning

The core retrieval component of the proposed system is a question-aware Graph Neural Network (GNN) designed to rank candidate answer entities within a local question-specific subgraph. Unlike traditional graph traversal approaches, which treat all relations uniformly, the proposed retriever conditions graph propagation on the semantic content of the input question. This allows the model to emphasize graph relations that are relevant to the question while suppressing unrelated structural information.

Formally, given a question-specific subgraph

$$G_q = (\mathcal{V}_q, \mathcal{E}_q),$$

the objective of the retriever is to estimate a relevance score

$$s(v, q)$$

for every node

$$v \in \mathcal{V}_q,$$

where higher scores indicate a greater likelihood that the node corresponds to a correct answer entity.

**Node Feature Initialization.** Each graph node corresponds to an entity in the knowledge graph and is initialized using a semantic embedding derived from the entity name. Rather than learning entity embeddings from scratch, the system encodes entity names using a pretrained text embedding model:

$$\mathbf{x}_v = \text{Embed}(t_v),$$

where:

- $t_v$  denotes the textual representation of entity  $v$ ,
- and  $\mathbf{x}_v \in R^d$  is the resulting embedding vector.

This initialization allows the retriever to incorporate semantic information directly from entity names while reducing dependence on purely structural graph features. The resulting embeddings are cached and reused during training and inference to avoid repeated computation.

**Question and Relation Representations.** The input question is encoded using the same embedding model:

$$\mathbf{q} = \text{Embed}(q),$$

where

$$\mathbf{q} \in R^d$$

represents the semantic embedding of the natural language question.

Relations are similarly represented through textual embeddings. Given a relation label

$$r \in \mathcal{R},$$

the system first converts the relation into a readable textual form and then computes:

$$\mathbf{r} = \text{Embed}(r).$$

Using a shared embedding space for questions and relations enables semantic matching between question intent and graph relations.

**Question-Relation Relevance.** A central component of the retriever is the computation of question-aware edge relevance. Given a question embedding  $\mathbf{q}$  and a relation embedding  $\mathbf{r}$ , the model estimates a relevance weight

$$\omega(q, r),$$

which measures how important relation  $r$  is for answering question  $q$ .

In the implemented system, relation relevance is computed using cosine similarity:

$$\omega(q, r) = \frac{\mathbf{q}^\top \mathbf{r}}{\|\mathbf{q}\| \|\mathbf{r}\|}.$$

This score acts as an attention-like weighting mechanism over graph edges. Relations semantically aligned with the question receive higher weights, while unrelated relations contribute less during message propagation.

**Question-Aware Message Passing.** After initialization, node representations are updated through multiple layers of weighted message passing. At each layer, nodes aggregate information from their neighbors while conditioning the aggregation on relation relevance.

Let

$$\mathbf{h}_v^{(k)}$$

denote the representation of node  $v$  at layer  $k$ . Message passing is defined as:

$$\mathbf{h}_v^{(k+1)} = \sigma \left( W_s \mathbf{h}_v^{(k)} + \sum_{u \in \mathcal{N}(v)} \omega(q, r_{uv}) \cdot W_m \mathbf{h}_u^{(k)} \right),$$

where:

- $\mathcal{N}(v)$  denotes the neighborhood of node  $v$ ,
- $r_{uv}$  is the relation connecting nodes  $u$  and  $v$ ,
- $\omega(q, r_{uv})$  is the question-aware edge weight,
- $W_s$  and  $W_m$  are trainable transformation matrices,
- and  $\sigma(\cdot)$  denotes a nonlinear activation function.

This formulation enables the model to propagate information preferentially through question-relevant graph paths. Neighboring nodes connected through highly relevant relations exert stronger influence on the updated representation, while messages traveling through irrelevant relations are attenuated.

By stacking multiple GNN layers, the retriever can capture multi-hop dependencies and aggregate evidence from increasingly distant portions of the graph. This is particularly important for WebQSP questions that require reasoning over chains of relations rather than direct one-hop connections.

**Answer Node Classification.** After the final message-passing layer, the model produces a representation for every node in the local subgraph. A multilayer perceptron (MLP) classifier is then applied independently to each node:

$$s(v, q) = \text{MLP}(\mathbf{h}_v^{(K)}),$$

where:



- $K$  denotes the number of GNN layers,
- and  $s(v, q)$  is the predicted answer score for node  $v$ .

The output score is transformed into a probability using the sigmoid function:

$$p(v \mid q) = \sigma(s(v, q)),$$

which represents the estimated probability that node  $v$  is a correct answer entity for question  $q$ .

Training is formulated as a binary node classification problem. Gold answer entities are labeled as positive nodes, while all remaining nodes are treated as negative nodes. Since local question subgraphs typically contain only a small number of answer entities compared to a large number of non-answer nodes, the training data is inherently class-imbalanced. Without correction, the model may become biased toward predicting negative labels due to their overwhelming frequency.

To mitigate this issue, a positive class weighting factor is introduced in the binary cross-entropy loss:

$$\mathcal{L} = -w_p y \log p - (1 - y) \log(1 - p),$$

where:

- $y \in \{0, 1\}$  denotes the ground-truth node label,
- $p$  is the predicted answer probability,
- and  $w_p$  is the positive class weight applied to answer nodes.

This weighting mechanism increases the contribution of positive examples during optimization and encourages the model to better distinguish correct answer entities from structurally similar but irrelevant neighboring nodes.

This question-aware retrieval mechanism forms the central reasoning component of the system. By integrating semantic question representations with graph-structured message passing, the retriever learns to identify compact sets of candidate answer entities while preserving the relational structure necessary for downstream reasoning and explanation.

### 3.4 Candidate Selection and Path Extraction

After graph-based message passing and node scoring, the retriever produces a probability estimate

$$p(v \mid q)$$

for every node

$$v \in \mathcal{V}_q,$$

representing the likelihood that the node corresponds to a correct answer entity for question  $q$ . The next stage of the pipeline transforms these node-level predictions into compact graph evidence suitable for downstream language-model reasoning.

**Candidate Answer Selection.** Rather than passing the full local graph to the language model, the system first filters the retrieved nodes and selects only the most relevant answer candidates.

Candidate selection is formulated as a constrained ranking problem over node scores. Let

$$\mathcal{C}_q$$

denote the final set of candidate answer entities for question  $q$ . The system first selects all nodes satisfying:

$$p(v \mid q) \geq \tau,$$

where:

- $p(v \mid q)$  is the predicted node probability,
- and  $\tau$  is a configurable confidence threshold.

However, depending on the question and graph structure, thresholding alone may produce either too few or too many candidate nodes. To stabilize retrieval quality and ensure sufficient evidence coverage, the system introduces both lower and upper candidate bounds.

First, a minimum candidate count  $K_{\min}$  is enforced. If fewer than  $K_{\min}$  nodes satisfy the confidence threshold, the retriever instead returns the top-ranked  $K_{\min}$  nodes according to their prediction scores. This prevents the reasoning stage from failing due to insufficient candidate coverage.

Second, an upper candidate limit  $K_{\max}$  is applied to avoid excessive graph expansion and prompt noise. If the number of selected candidates exceeds  $K_{\max}$ , only the highest-ranked  $K_{\max}$  nodes are retained.

Consequently, the final candidate set is constrained as:

$$K_{\min} \leq |\mathcal{C}_q| \leq K_{\max}.$$

This hybrid threshold-and-ranking strategy balances precision and coverage by ensuring that:

- highly confident nodes are prioritized,
- retrieval remains robust even when confidence scores are uncertain,
- and the reasoning context remains compact enough for efficient LLM inference.

**Shortest-Path Extraction.** After candidate selection, the system extracts graph paths connecting the question entities to the retrieved candidate answers. These paths serve as explicit reasoning evidence for downstream answer generation.

Given a set of question entities

$$S_q \subseteq \mathcal{V}_q$$

and a candidate answer node

$$c \in \mathcal{C}_q,$$

the objective is to identify short relational paths:

$$\pi(s, c), \quad s \in S_q,$$

that connect the question entities to the candidate node.

Shortest paths are computed using Breadth-First Search (BFS) over the graph structure. BFS is particularly suitable because:

- the graph is unweighted,
- shortest relational explanations are generally more interpretable,
- and short paths reduce prompt complexity for the language model.

The retrieved paths therefore approximate concise relational explanations for why a candidate node was selected by the retriever.

**Handling Multiple Shortest Paths.** In many cases, multiple shortest paths of equal length may exist between the question entity and a candidate answer node. Restricting reasoning to only a single path may discard useful relational evidence or introduce bias toward arbitrary traversal choices.

To preserve richer graph context, the system retains all shortest paths whose lengths equal the minimum discovered distance:

$$|\pi_i| = |\pi_j| = d_{\min}.$$

This allows the reasoning stage to access multiple relational justifications for the same candidate entity, improving evidence coverage while still preserving compactness.

**Reasoning Subgraph Construction.** The extracted shortest paths are merged into a unified reasoning subgraph:

$$G_q^{\text{reason}} = (\mathcal{V}_q^{\text{reason}}, \mathcal{E}_q^{\text{reason}}).$$

Formally, the reasoning subgraph is constructed as the union of all retrieved path edges:

$$\mathcal{E}_q^{\text{reason}} = \bigcup_{\pi \in \Pi_q} \mathcal{E}(\pi),$$

where:

- $\Pi_q$  denotes the set of retrieved shortest paths,
- and  $\mathcal{E}(\pi)$  denotes the edge set belonging to path  $\pi$ .

Duplicate triples are removed during this process, producing a compact relational structure that contains only the graph evidence relevant to the selected candidates.

This reasoning subgraph serves as the final structured context passed to the language model. Instead of reasoning over the full graph or over isolated candidate nodes, the LLM receives a compact graph-grounded explanation that preserves explicit relational connectivity between question entities and retrieved answers.

### 3.5 Extended Retriever Architecture

In addition to the baseline retriever architecture described previously, the framework also supports a more advanced experimental configuration referred to as **GNN-AA** (Graph Neural Network – Advanced Architecture). This configuration extends the baseline retriever with several additional mechanisms intended to improve question-aware graph reasoning, edge conditioning, and training stability.

The complete design and implementation details of the GNN-AA configuration are provided in Appendix A.

### 3.6 LLM-Based Answer Generation

Although graph neural networks are effective at retrieving relevant entities and reasoning over graph structure, large language models operate primarily over textual input. Consequently, the extracted graph evidence must first be transformed into a textual representation before it can be used for downstream answer generation.

**Graph-to-Text Transformation** The reasoning subgraph is converted into textual relational paths connecting the question entities to the retrieved candidate answers. Each shortest path is represented as an ordered sequence of entities and relations:

$$v_0 \rightarrow r_1 \rightarrow v_1 \rightarrow \cdots \rightarrow r_n \rightarrow v_n.$$

Each extracted path is provided sequentially as textual evidence. When multiple shortest paths exist for the same candidate answer, all paths are included in the context in order to preserve alternative relational explanations.

This graph-to-text transformation preserves explicit graph connectivity while making the structured evidence directly consumable by the language model.

**Prompt Construction.** The final LLM prompt consists of:

- the original natural language question,
- and the textualized reasoning subgraph extracted from the graph retriever.

Conceptually, the prompt follows the structure:

**Question + Reasoning Paths.**

The language model is instructed to answer the question using only the provided graph evidence. Restricting the model to explicit retrieved paths helps reduce hallucinations and grounds the generated answer in the relational structure of the knowledge graph.

**Answer Generation and Explanation.** Given the question and reasoning context, the language model generates:

- the final predicted answer,

- and a short explanation describing the graph evidence used for the prediction.

The explanation corresponds to the reasoning path or paths that most strongly support the generated answer. This design improves interpretability by allowing the final prediction to be traced back to explicit graph relations rather than relying solely on implicit language-model reasoning.

The overall generation stage therefore performs a graph-to-text-to-answer transformation:

Graph Evidence  $\rightarrow$  Textual Reasoning Context  $\rightarrow$  Generated Answer.

By separating graph retrieval from language generation, the system combines the structural reasoning capabilities of graph neural networks with the natural language understanding and explanation capabilities of large language models. The resulting pipeline preserves explicit relational evidence while enabling flexible and interpretable answer generation.

## 4 Experimental Setup

### 4.1 Retriever Configuration

The retrieval component is implemented as a question-aware weighted Graph Neural Network (GNN) operating over local WebQSP subgraphs. The retriever receives a question-specific graph together with semantic embeddings of entities, questions, and relations, and produces a relevance score for every node in the graph.

**Embedding Configuration.** Entity names, questions, and relation labels are embedded using OpenAI text embedding models. Two embedding configurations are supported:

- `text-embedding-3-small`,
- `text-embedding-3-large`.

The default configuration uses `text-embedding-3-small`, which produces embedding vectors of dimension 1536. The system also supports the larger `text-embedding-3-large` model, which produces embedding vectors of dimension 3072.

All embeddings are cached locally and reused during both training and inference in order to reduce repeated API computation and improve experimental reproducibility. The embedding cache is implemented using the Qdrant vector database, where each embedding vector is stored together with metadata describing the dataset, embedding model, cache type, and original textual input. Separate collections are maintained for different embedding configurations, and vectors are indexed using cosine similarity. This design enables efficient embedding reuse, fast lookup of previously computed vectors, and scalable management of entity, relation, and question embeddings across experiments.

**GNN Architecture.** The retriever architecture consists of:

1. an input projection layer,

| Component           | Supported values                                                           | Default                             |
|---------------------|----------------------------------------------------------------------------|-------------------------------------|
| Embedding model     | <code>text-embedding-3-small</code><br><code>text-embedding-3-large</code> | <code>text-embedding-3-small</code> |
| Embedding dimension | 1536, 3072                                                                 | 1536                                |

Table 1: Embedding configuration options.

2. multiple weighted message-passing layers,
3. and a node-level classifier.

The overall computation pipeline is:

Embeddings  $\rightarrow$  Projection  $\rightarrow$  GNN Layers  $\rightarrow$  Classifier  $\rightarrow$  Node Scores.

Entity embeddings are first projected into the hidden representation space:

$$\mathbf{h}_v^{(0)} = W_p \mathbf{x}_v,$$

where:

- $\mathbf{x}_v$  is the original entity embedding,
- and  $W_p$  is a trainable linear projection matrix.

The system supports both two-layer and three-layer GNN configurations, with hidden dimensions of: 128, 256 and 512.

| Component        | Supported values                       | Default          |
|------------------|----------------------------------------|------------------|
| GNN layers       | 2, 3                                   | 2                |
| Hidden dimension | 128, 256, 512                          | 256              |
| Node classifier  | <code>mlp</code> , <code>linear</code> | <code>mlp</code> |

Table 2: Retriever architecture configuration.

**Weighted Message Passing.** The retriever uses weighted message passing conditioned on question-relation relevance scores, as described in the solution architecture section. In the implemented model, edge weights are computed using cosine similarity between the question embedding and the corresponding relation embedding:

$$\omega(q, r) = \frac{\mathbf{q}^\top \mathbf{r}}{\|\mathbf{q}\| \|\mathbf{r}\|}.$$

The weighted messages are then aggregated into the destination node:

$$\mathbf{a}_v^{(k)} = \sum_{u \in \mathcal{N}(v)} \omega(q, r_{uv}) \cdot W_m \mathbf{h}_u^{(k)}.$$

The aggregated message is concatenated with the current node state:

$$[\mathbf{h}_v^{(k)} \parallel \mathbf{a}_v^{(k)}],$$

followed by a linear update projection and ReLU activation:

$$\mathbf{h}_v^{(k+1)} = \text{ReLU} \left( W_u [\mathbf{h}_v^{(k)} \parallel \mathbf{a}_v^{(k)}] \right).$$

Consequently, each GNN layer contains:

- a message projection:

$$\text{hidden\_dim} \rightarrow \text{hidden\_dim},$$

- an update projection:

$$2 \cdot \text{hidden\_dim} \rightarrow \text{hidden\_dim},$$

- and a ReLU activation function.

**Node Classification and Optimization.** After the final GNN layer, the retriever predicts one logit for every node in the local graph:

$$s(v, q) = \text{Classifier}(\mathbf{h}_v^{(K)}).$$

Two classifier variants are supported:

- a linear classifier,
- and a multilayer perceptron (MLP).

The default configuration uses the MLP classifier.

Training is formulated as binary node classification using: `BCEWithLogitsLoss`. Since local question subgraphs contain significantly more negative nodes than positive answer nodes, the training process applies dynamic positive class weighting:

$$w_p = \frac{N_{\text{negative}}}{N_{\text{positive}}},$$

where:

- $N_{\text{negative}}$  is the number of non-answer nodes,
- and  $N_{\text{positive}}$  is the number of gold answer nodes.

This weighting mechanism increases the contribution of sparse positive examples during optimization and reduces bias toward predicting negative labels.

The default optimization configuration uses the Adam optimizer with a learning rate of  $10^{-3}$ , no weight decay, and a total of 3 training epochs.

**Candidate Selection Configuration.** Candidate answer entities are selected using the hybrid threshold-and-ranking strategy described in the retrieval architecture section. During experimentation, multiple threshold values and candidate-limit configurations were evaluated,

including substantially larger candidate limits than the default setup, in order to analyze the trade-off between retrieval precision, graph coverage, prompt size, and downstream reasoning efficiency.

Table 3 summarizes the default configuration used in the reported experiments.

| Parameter                   | Default value |
|-----------------------------|---------------|
| Confidence threshold $\tau$ | 0.5           |
| Minimum top- $K$ candidates | 10            |
| Candidate limit             | 15            |

Table 3: Default candidate selection configuration.

## 4.2 LLM Reasoning Configuration

The final answer generation stage is implemented using large language models conditioned on the textualized reasoning subgraphs extracted by the retriever. Since the reasoning stage operates over graph-derived relational paths rather than short isolated prompts, inference becomes highly input-token intensive. Even relatively small experimental runs over approximately 100 training and evaluation instances may consume on the order of several hundred thousand input tokens, with typical runs reaching roughly: 700,000 input tokens.

As a result, inference cost becomes an important practical consideration, particularly during iterative experimentation, hyperparameter tuning, and prompt refinement. For this reason, the experimental setup prioritizes smaller and more cost-efficient reasoning models during early-stage evaluation, with larger models reserved for later experiments and comparative analysis.

The framework supports multiple OpenAI and DeepSeek language models. Table 4 summarizes the supported models together with the subset used in the reported experiments.

| Model family | Supported models        | Used in experiments |
|--------------|-------------------------|---------------------|
| GPT-5.4      | 5.4, 5.4-mini, 5.4-nano | 5.4-mini, 5.4-nano  |
| GPT-5        | 5-mini, 5-nano          | 5-mini              |
| GPT-4.1      | 4.1, 4.1-mini, 4.1-nano | 4.1-mini, 4.1-nano  |
| DeepSeek-V4  | v4-flash, v4-pro        | v4-pro              |

Table 4: Supported LLM configurations and models used in experiments.

These models provide a practical balance between reasoning quality, latency, and inference cost. In particular, the DeepSeek-V4-Pro model offers relatively strong reasoning performance while remaining significantly cheaper than larger frontier-scale alternatives, making it suitable for large-scale retrieval-augmented graph reasoning experiments.

## 4.3 Evaluation Metrics

The evaluation separates retrieval quality from final answer quality. Retrieval metrics evaluate whether the GNN retrieves correct answer candidates, while final answer metrics evaluate the output produced by the LLM.



**Retrieval evaluation.** Hits@1 and Hits@candidate limit evaluate whether the GNN ranks gold answer nodes near the top.

**Final answer evaluation.** Generated and gold answers are normalized before matching. Hit rate, precision, recall, and F1 evaluate the final LLM output. Failed, empty, and **Unknown** outputs are treated as no prediction.

**Explanation and ranking evaluation.** Grounding metrics measure whether generated explanation triples are supported by the reasoning subgraph. nDCG metrics summarize ranking quality at fixed cutoffs and at the configured candidate limit.

| Retrieval metric     | Definition                                            |
|----------------------|-------------------------------------------------------|
| Hits@1               | Top retrieved candidate is a gold answer.             |
| Hits@candidate limit | A gold answer appears within the retained candidates. |

Table 5: Retrieval and coverage metrics.

| Answer metric             | Definition                                          |
|---------------------------|-----------------------------------------------------|
| Hit rate                  | At least one generated answer is correct.           |
| Precision                 | Share of predicted answers that are correct.        |
| Recall                    | Share of gold answers recovered.                    |
| F1                        | Harmonic mean of precision and recall.              |
| Grounded explanation rate | At least one explanation triple is in the subgraph. |
| nDCG@10                   | Ranking quality at fixed cutoffs.                   |

Table 6: Final answer, explanation-grounding, and ranking metrics.

## 5 Results

This section presents the experimental results across multiple retriever and reasoning-model configurations. All retrievers were trained on 300 WebQSP training instances and evaluated on 100 test instances. Every experiment used a confidence threshold of  $\tau = 0.65$ , minimum candidate count  $K = 10$ , candidate limit of 55, and 10 training epochs.

The evaluated configurations include both the baseline retriever architecture and the advanced GNN-AA architecture introduced in Appendix A.

To simplify result presentation, experiment names encode the main retriever and reasoning-model configuration. For example, **GNN-3-256-gpt-4.1-mini** denotes the baseline GNN retriever with 3 message-passing layers, a hidden dimension of 256, and **gpt-4.1-mini** used for answer generation. Similarly, configurations prefixed with **GNN-AA** correspond to the advanced retriever architecture described in Appendix A.

**Retriever Performance.** Figure 1 presents Hit@CandidateLimit across all evaluated configurations. The strongest retrieval coverage was achieved by **GNN-3-512-gpt-5.4-mini**, which obtained a score of 0.78. The **GNN-3-512-deepseek-v4-pro** configuration followed closely with a score of 0.71.

Among the advanced retrievers, **GNN-AA-256-gpt-5.4-mini** achieved competitive retrieval coverage with a score of 0.69, while **GNN-AA-512-gpt-5.4-mini** performed substantially worse with a score of 0.41. This indicates that the advanced architecture was sensitive to larger hidden dimensions and produced less stable retrieval behavior in the evaluated setting.

Interestingly, the Hits@1 results shown in Figure 2 differ from the broader retrieval trends. The **GNN-AA-512-gpt-5.4-mini** configuration achieved the highest Hits@1 score of 0.09 despite having substantially lower retrieval coverage overall. This suggests that the advanced architecture occasionally ranks the correct answer at the top position while failing to consistently preserve broader candidate coverage.

Ranking quality measured using nDCG@10 is shown in Figure 3. The strongest ranking quality was achieved by **GNN-3-512-gpt-5.4-mini**, which obtained an nDCG@10 score of 0.18. The remaining configurations achieved noticeably lower ranking quality, particularly the GNN-AA variants.

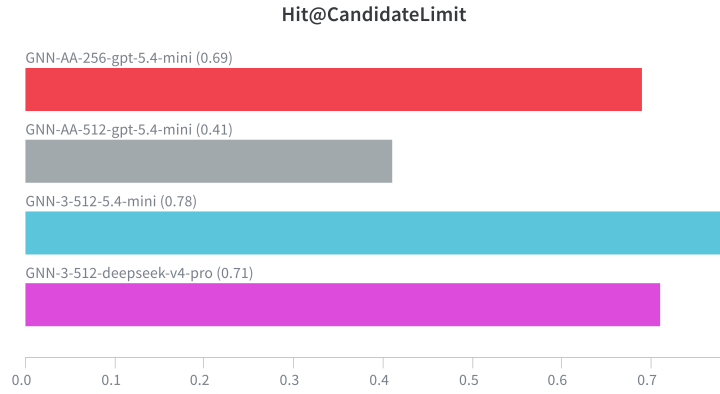


Figure 1: Hit@CandidateLimit across evaluated configurations.

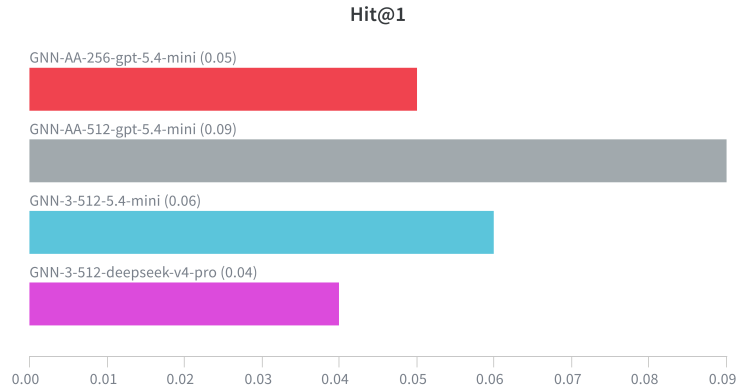


Figure 2: Hits@1 across evaluated configurations.

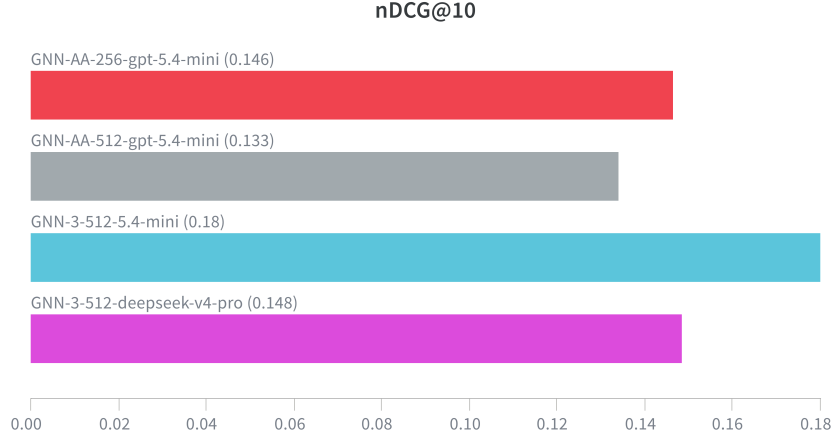


Figure 3: nDCG@10 across evaluated configurations.

**Answer Generation Performance** Final answer quality was evaluated using hit rate, precision, recall, and F1 over generated answers.

The overall answer hit-rate results are shown in Figure 4. The strongest performance was achieved by **GNN-3-512-gpt-5.4-mini**, which obtained a hit rate of 0.68, closely followed by **GNN-3-512-deepseek-v4-pro** with 0.65. These results indicate that both baseline 3-layer retrievers were highly effective at retrieving reasoning evidence sufficient for producing at least one correct answer.

The advanced GNN-AA configurations achieved lower hit-rate performance overall. The **GNN-AA-256-gpt-5.4-mini** configuration obtained a score of 0.61, while **GNN-AA-512-gpt-5.4-mini** dropped further to 0.51. This behavior is consistent with the retrieval observations discussed previously, where the advanced architecture produced more selective candidate retrieval behavior and lower overall candidate coverage.

F1 results are shown in Figure 5. The strongest overall answer quality was achieved by **GNN-3-512-deepseek-v4-pro**, which obtained an F1 score of 0.195. The **GNN-3-512-gpt-5.4-mini** configuration remained competitive with an F1 score of 0.172.

Precision results are shown in Figure 7. The **GNN-3-512-deepseek-v4-pro** configuration achieved the highest precision with a score of 0.831, outperforming all remaining configurations. The baseline **GNN-3-512-gpt-5.4-mini** configuration achieved a precision score of 0.748.

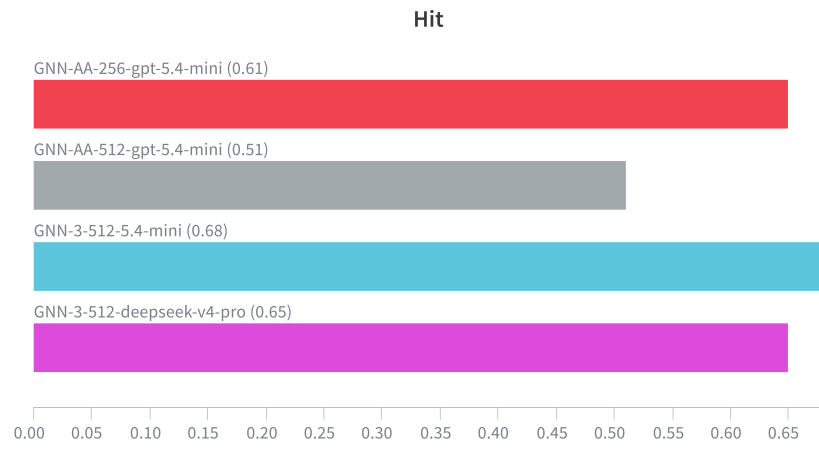


Figure 4: Final answer hit rate across evaluated configurations.

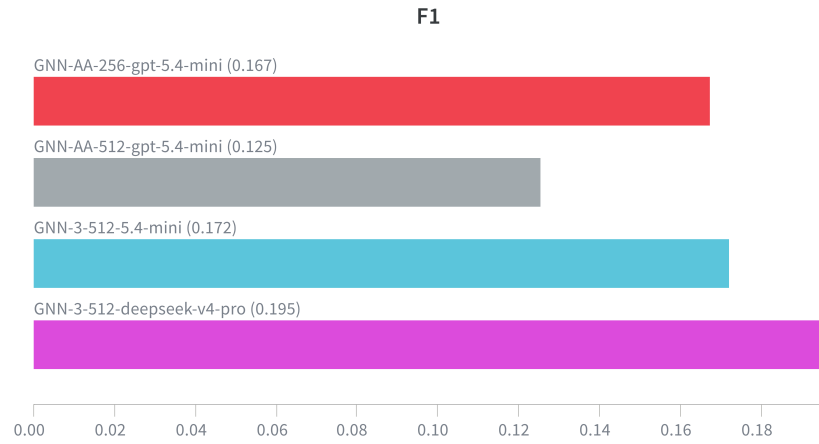


Figure 5: F1 scores across evaluated configurations.

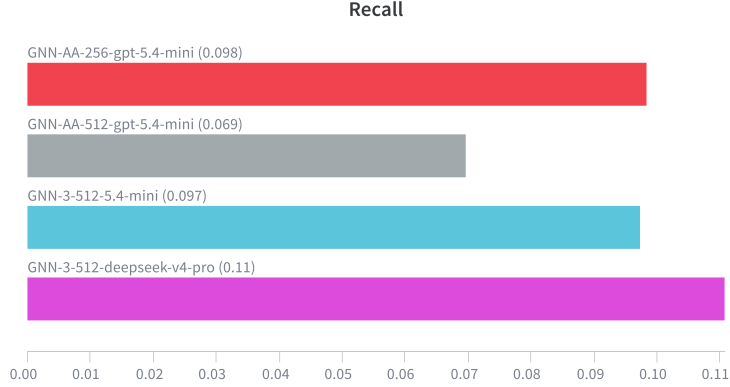


Figure 6: Recall across evaluated configurations.

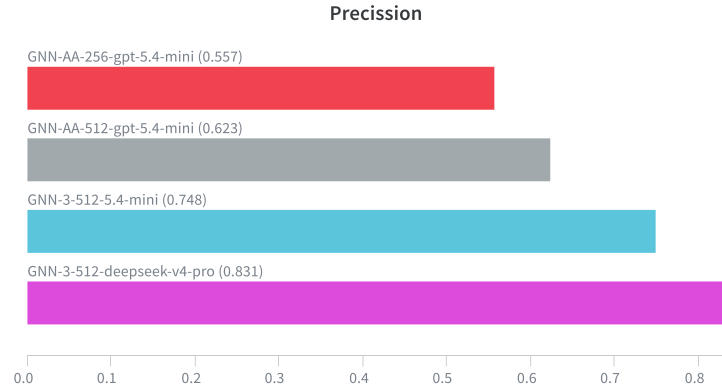


Figure 7: Precision across evaluated configurations.

Recall results are presented in Figure 6. Again, **GNN-3-512-deepseek-v4-pro** achieved the strongest performance with a recall score of 0.11, while both advanced GNN-AA configurations produced lower recall values.

Overall, the results suggest that although GPT-5.4 Mini achieved slightly stronger retrieval ranking behavior, DeepSeek-V4-Pro utilized retrieved graph evidence more effectively during downstream answer generation, producing stronger precision, recall, and overall F1 performance.

**Explanation Grounding.** Grounded explanation rate results are shown in Figure 8. The **GNN-3-512-deepseek-v4-pro** configuration substantially outperformed the remaining models with a grounding score of 0.17.

This result indicates that broader retrieval coverage combined with stronger reasoning consistency improves the likelihood that generated explanations remain supported by triples present in the retrieved reasoning subgraph.

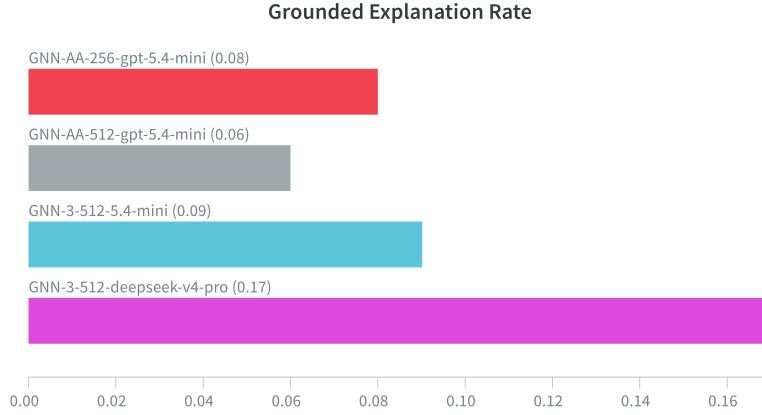


Figure 8: Grounded explanation rate across evaluated configurations.

**Candidate Retrieval Behavior.** Figure 9 presents the average number of retrieved candidates for each configuration.

The **GNN-3-512-deepseek-v4-pro** configuration produced the largest candidate sets with an average of 50.01 retrieved candidates, while **GNN-AA-512-gpt-5.4-mini** retrieved only 28.37 candidates on average.

This behavior strongly correlates with the observed recall and grounding metrics. Larger candidate sets preserve more relational evidence and improve downstream reasoning support, while smaller candidate sets reduce graph coverage and explanation grounding.

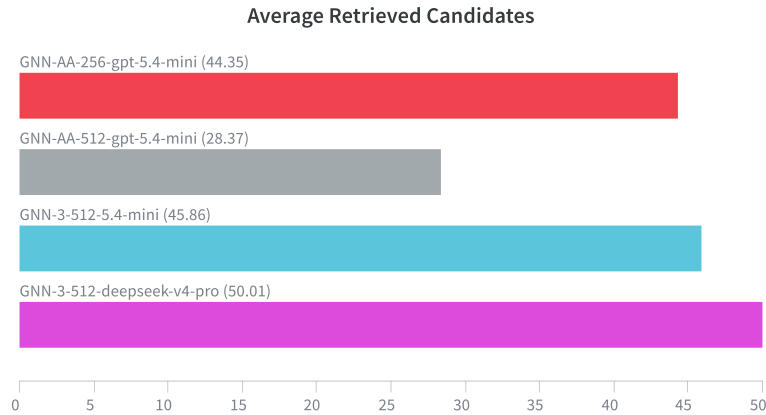


Figure 9: Average retrieved candidate count across evaluated configurations.

**Token Usage and Inference Cost.** Finally, Figure 10 presents total token usage across all evaluated configurations.

The `GNN-AA-256-gpt-5.4-mini` configuration produced the highest token usage with approximately 1.3 million tokens. In contrast, `GNN-AA-512-gpt-5.4-mini` achieved the lowest token usage with approximately 916 000 tokens.

These results highlight an important trade-off in graph-based retrieval-augmented reasoning systems: larger retrieved reasoning subgraphs improve retrieval coverage and explanation grounding, but substantially increase inference cost and prompt complexity.

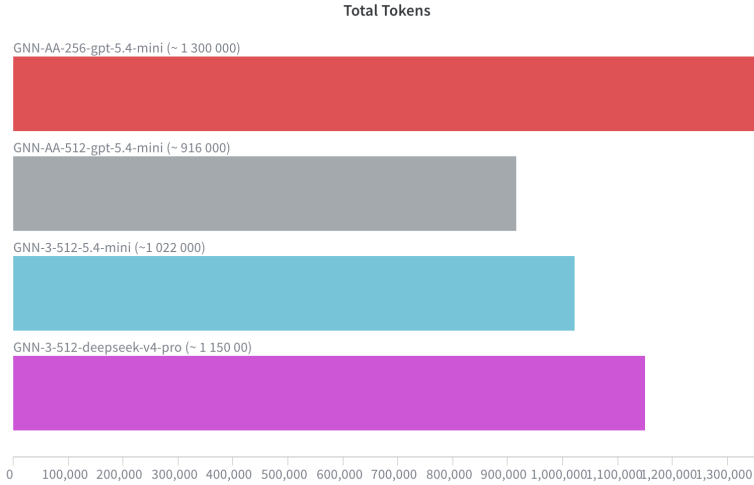


Figure 10: Total token usage across evaluated configurations.

## 6 Limitations and Future Work

The current pipeline is mainly tailored to WebQSP, which limits its generality. Future work should support more datasets and provide more configurable options for graph construction, retrieval, path extraction, embedding models, and LLM backends.

The system also depends on the quality of the WebQSP local graphs. If a gold answer is missing from the graph, the retriever cannot recover it. Improving graph coverage or using external knowledge sources could reduce this limitation.

Another limitation is entity mapping. Knowledge-graph nodes are often represented by identifiers, which must be mapped to readable entity names before being useful for the LLM. The current system relies on a stored mapping file between node identifiers and entity names. Future work should improve this mapping process, handle missing or ambiguous mappings more robustly, and support richer entity metadata.

Model capacity is also a limitation. More precise and expensive LLMs could improve final answer generation, while more complex GNN architectures could improve candidate ranking and retrieval quality.

Finally, training is limited by available computational resources. Training on more samples, testing larger models, and running broader hyperparameter searches are important directions for future work.

## 7 Conclusion

In this paper, we introduce GraphRagX, a modular framework that combines question-aware graph neural retrieval with large language model reasoning for knowledge graph question answering. By separating candidate retrieval, reasoning-path extraction, and answer generation, the framework enables graph-grounded reasoning while preserving interpretable relational evidence.

Experiments on WebQSP demonstrate that the framework can effectively retrieve relevant answer candidates and provide structured graph evidence for downstream language-model reasoning. The results highlight the importance of retrieval quality for final answer generation and explanation grounding, demonstrating the value of combining graph neural networks and large language models in a unified retrieval-augmented KGQA pipeline.

Overall, GraphRagX provides a flexible foundation for interpretable and evidence-grounded knowledge graph question answering and offers a practical framework for future research on graph-aware retrieval and reasoning systems.

## References

- [1] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh, “The value of semantic parse labeling for knowledge base question answering,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, K. Erk and N. A. Smith, Eds. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 201–206. [Online]. Available: <https://aclanthology.org/P16-2033/>
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2021. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [3] H. Sun, T. Bedrax-Weiss, and W. Cohen, “PullNet: Open domain question answering with iterative retrieval on knowledge bases and text,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 2380–2390. [Online]. Available: <https://aclanthology.org/D19-1242/>
- [4] A. Saxena, A. Tripathi, and P. Talukdar, “Improving multi-hop question answering over knowledge graphs using knowledge base embeddings,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 4498–4507. [Online]. Available: <https://aclanthology.org/2020.acl-main.412/>



- [5] J. Shi, S. Cao, L. Hou, J. Li, and H. Zhang, “TransferNet: An effective and transparent framework for multi-hop question answering over relation graph,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 4149–4158. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.341/>
- [6] C. Mavromatis and G. Karypis, “ReaRev: Adaptive reasoning for question answering over knowledge graphs,” in *Findings of the Association for Computational Linguistics: EMNLP 2022*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 2447–2458. [Online]. Available: <https://aclanthology.org/2022.findings-emnlp.181/>
- [7] —, “Gnn-rag: Graph neural retrieval for large language model reasoning,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.20139>
- [8] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, “G-retriever: Retrieval-augmented generation for textual graph understanding and question answering,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.07630>
- [9] F. Wu, V. P. Dwivedi, and J. Leskovec, “Large language models are good relational learners,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.05725>

## A Advanced Retriever Architecture (GNN-AA)

The GNN-AA configuration extends the baseline retriever with several optional architectural upgrades designed to improve question-aware reasoning and graph propagation.

**Trainable Edge Relevance Scoring.** In the baseline retriever, edge relevance scores are computed using cosine similarity between question and relation embeddings. GNN-AA additionally supports a trainable edge-scoring module implemented as a multilayer perceptron (MLP). The edge-scoring network receives the concatenation of:

$$[\mathbf{q}, \mathbf{r}, \mathbf{q} \odot \mathbf{r}],$$

where:

- $\mathbf{q}$  is the projected question embedding,
- $\mathbf{r}$  is the projected relation embedding,
- and  $\odot$  denotes element-wise multiplication.

The resulting edge score is transformed through a sigmoid activation to produce learned edge weights used during message propagation.

**Question-Aware Node Classification.** The advanced architecture additionally supports question-aware node classification. Instead of predicting answer logits solely from the final node representation, the classifier jointly conditions predictions on both the node embedding and the question embedding.

The classifier input is constructed as:

$$[\mathbf{h}_v, \mathbf{q}, \mathbf{h}_v \odot \mathbf{q}],$$

allowing the classifier to explicitly model interactions between node semantics and question semantics during answer prediction.

**Reverse Edge Propagation.** To improve relational coverage during message passing, GNN-AA optionally augments the graph with reverse edges. For every original graph relation:

$$u \xrightarrow{r} v,$$

a reverse relation is added:

$$v \xrightarrow{r^{-1}} u.$$

Reverse relations are assigned independent embeddings and participate in message propagation identically to the original graph edges. This allows information to flow bidirectionally through the graph and improves reasoning over asymmetric relation structures.

**Residual Connections and Layer Normalization.** The advanced architecture additionally supports residual connections and layer normalization after each message-passing layer. Given the message output:

$$\mathbf{m}^{(k)},$$

the updated node representation is computed as:

$$\mathbf{h}^{(k+1)} = \text{ReLU} \left( \text{LayerNorm}(\mathbf{h}^{(k)} + \text{Dropout}(\mathbf{m}^{(k)})) \right).$$

This residual normalization strategy improves optimization stability and helps preserve information across multiple graph propagation layers.